

# Central Inteligente de Monitoramento

## Integrando APIs para automatizar processos e gerar insights

Disciplina de Integração de APIs — UniFECAF

Aplicação: Central Lumen

O Instituto Lumen é uma instituição fictícia criada para este trabalho. Todos os perfis de aluno são sintéticos, gerados por script versionado com semente fixa. Nenhum dado corresponde a pessoa real.

## 1. Contextualização do problema

Uma escola de formação continuada opera com a informação dividida entre pelo menos três lugares. O cadastro do aluno — turma, progresso, presença em mentorias, prazo de acesso — vive num sistema de gestão. As regras que dizem o que fazer com esse cadastro vivem num documento que a coordenação escreveu. A decisão de aplicar uma coisa à outra vive na cabeça de quem atende.

O diagnóstico usual dessa situação é "falta de integração", e a métrica usual é tempo perdido trocando de aba. Esse diagnóstico é raso e leva à solução errada, que é um dashboard de leitura.

O custo real é outro: **a regra e o dado nunca se encontram sozinhos**. Ninguém varre a base inteira toda semana cruzando cada aluno contra o playbook. O cruzamento só acontece quando alguém pergunta — e quem está prestes a perder o prazo por um único critério normalmente não sabe que deveria perguntar.

Isso produz três falhas concretas:

**A regra envelhece sem que ninguém perceba.** Quando a regra é reimplementada em código ou em SQL, editar o documento deixa de mudar o comportamento. As duas versões divergem, e a divergência só aparece numa reclamação.

**O caso limítrofe é decidido por intuição.** A regra de extensão de acesso exige no mínimo 4 mentorias do CARPA e 3 de Pilares. Um aluno com 4 e 2 tem 50% do total de encontros. Perguntado informalmente, o atendimento tende a responder "você está quase lá" — porque em linguagem natural 50% soa como quase. A regra define pisos por fase justamente porque as fases não são substituíveis, e a intuição de média destrói essa informação.

**O trabalho não é criado, só respondido.** Um sistema reativo atende quem procura. Quem desengajou, por definição, não procura.

## 2. Descrição da solução proposta

A **Central Lumen** consolida as fontes numa aplicação e, principalmente, **inverte a iniciativa**: em vez de esperar a pergunta, varre a base, aplica as regras e abre trabalho atribuído a um responsável.

A arquitetura se resume a uma frase: *a regra vem do Notion, a conta é feita em código, e o modelo só redige.*

| Camada            | Responsabilidade                            | O que ela não faz  |
|-------------------|---------------------------------------------|--------------------|
| Notion · Playbook | guarda o texto da regra, com os limiares    | não calcula        |
| Motor de regras   | faz a aritmética e registra a conta literal | não define a regra |
| Gemini            | redige a leitura para a coordenação         | <b>não decide</b>  |
| Notion · Alertas  | registra o caso como trabalho atribuído     | não guarda regra   |

Cada fronteira dessa tabela responde a uma falha da seção anterior.

**Contra o envelhecimento da regra:** o texto da regra é lido do Notion em execução, e os limiares numéricos são extraídos do próprio texto — não são constantes no código. Editar "no mínimo 4 mentorias do CARPA" para 5 no Notion muda o comportamento na chamada seguinte. Quando um limiar não está declarado no texto, o sistema usa um padrão e **marca isso na conta**, para que um alerta sustentado por padrão em vez de por texto seja visivelmente mais frouxo.

**Contra a intuição de média:** a avaliação é determinística e mantém os critérios separados. A saída registra a conta literal e o veredito:

```
Mentorias do CARPA: 4 de 5 – mínimo 4, cumpre
Mentorias de Pilares: 2 de 7 – mínimo 3, NÃO cumpre
Módulos faltantes: 2 – no máximo 2, cumpre
Reprovado por um único critério (Mentorias de Pilares) – os pisos são
independentes e não se compensam entre si.
```

O veredito conta quantos critérios falharam em vez de usar frase fixa. "Um único critério" é o argumento contra o "mas ele está em 50%, quase lá"; aplicado a um aluno que falhou em três, descreveria mal a situação e gastaria, no caso banal, a força que o argumento precisa ter no caso limítrofe.

O teste automatizado que garante o cálculo existe porque a implementação errada — somar as fases e comparar com 60% — passa em todos os outros casos e falha só nesse.

**Contra a reatividade:** a varredura percorre a base inteira sem que ninguém tenha perguntado por um aluno específico. É essa varredura, e não o gatilho que a dispara, que inverte a iniciativa: ela cria trabalho em vez de responder a quem procurou.

Há uma quarta decisão, sobre o papel do modelo. O Gemini poderia avaliar a elegibilidade: tem a regra e tem os números. Ele não avalia porque duas execuções sobre o mesmo aluno poderiam divergir, e não haveria como auditar qual estava certa. O modelo recebe a conta **já resolvida** e é instruído a não recalcular. A conta determinística e a leitura gerada ficam lado a lado na mesma página, e o alerta declara que, havendo divergência, a conta prevalece.

### 3. APIs utilizadas e justificativa

#### Notion API ([api.notion.com/v1](https://api.notion.com/v1))

Faz dois papéis, e é a dupla função que justifica a escolha em vez de Airtable.

**Como banco de dados no-code.** Três bases relacionadas nativamente:

| Base             | Papel                                     |
|------------------|-------------------------------------------|
| Lumen · Alunos   | registro operacional sintético, 40 perfis |
| Lumen · Playbook | as regras, uma por página                 |
| Lumen · Alertas  | saída da automação                        |

**Como fila de trabalho atribuída.** O campo *Responsável* é do tipo pessoa, e a base funciona como fila: cada caso tem dono, severidade, prazo e status.

Duas justificativas, uma organizacional e uma técnica.

*Organizacional:* a coordenação já trabalha em Notion. Uma regra que vive onde a coordenação já escreve é uma regra que ela mantém. Uma regra que vive num banco que ela não abre vira documentação morta em duas semanas — e aí o sistema volta a ter regra desatualizada, que é a falha nº 1 da seção anterior.

*Técnica:* o Notion entrega, sem serviço adicional, a estrutura que o caso de uso exige — relação entre bases, campo de pessoa, status e prazo. A alternativa para a parte de atribuição seria integrar um provedor de e-mail ou um webhook, o que somaria uma credencial, uma fila de retentativa e **um segundo lugar por onde dado de aluno trafega**. Reduzir a superfície de tratamento é um objetivo de governança, não só economia de código.

**Correção a uma versão anterior deste documento.** A justificativa técnica original dizia que atribuir o *Responsável* dispara a notificação nativa do Notion. A afirmação foi escrita a partir do comportamento documentado da ferramenta, e **não se confirmou em teste**. Está corrigida aqui em vez de apagada, porque a causa é instrutiva. Ver a seção 5.

*O que se perde:* o Notion não é um banco relacional. Não há transação, não há constraint de integridade, e a paginação é de 100 itens. A escolha é defensável nesta escala e seria questionável em ordens de grandeza acima.

**Autenticação:** token `Bearer` no header, com a versão da API fixada em `2022-06-28` — deixar sem fixar significa aceitar mudança de contrato num deploy que ninguém fez.

O token em uso é um **Personal Access Token**, e a distinção é material para a seção 5. Um PAT age com as permissões do usuário que o emitiu, sobre tudo que esse usuário enxerga no workspace. Uma integração interna, ao contrário, só alcança as páginas explicitamente compartilhadas com ela. As duas coisas usam o mesmo header e o mesmo código; o escopo é radicalmente diferente.

## Google Gemini API ([generativelanguage.googleapis.com/v1beta](https://generativelanguage.googleapis.com/v1beta))

Dois usos, ambos de redação:

1. **Leitura da coordenação** no alerta: transforma a conta resolvida em duas a quatro frases acionáveis, com instrução explícita de não recalcular e de não somar mentorias de fases diferentes.
2. **Pergunta ao Playbook** no painel: responde em linguagem natural usando só as regras carregadas, e devolve qual regra citou.

*Por que sem busca vetorial:* o Playbook tem cinco regras e cabe inteiro no prompt. Recuperação por embeddings nessa escala adiciona um passo que pode errar sem ganho de precisão. Se o Playbook crescer além da janela de contexto, a recuperação passa a valer o custo — mas otimizar antes disso seria complexidade sem problema correspondente.

**Autenticação:** chave de API no header `x-goog-api-key`, guardada como segredo da Edge Function.

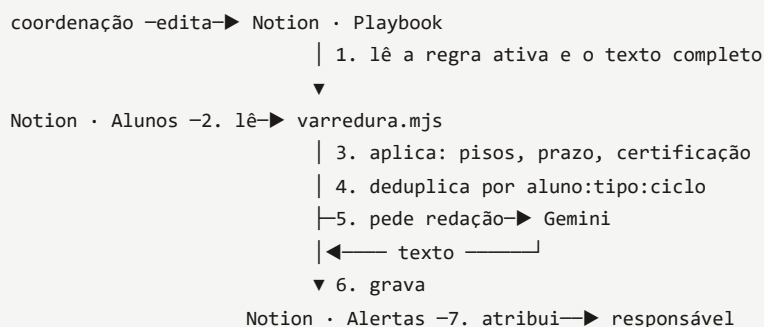
## Supabase — infraestrutura, não integração

Não é contado como uma das APIs integradas, porque não é fonte de informação de negócio. Entra por dois motivos: **autenticação** e **trilha de auditoria**.

Não guarda dado de aluno. Duplicar o registro criaria duas fontes da verdade que divergem na primeira vez que a coordenação editar uma delas.

## 4. Fluxo de integração

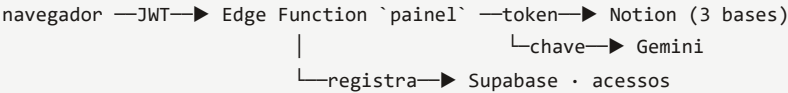
### Fluxo de escrita — a automação



O passo 4 não é detalhe. Sem deduplicação, uma varredura diária reabre o mesmo alerta todo dia, a base vira ruído e a coordenação para de olhar — o que faz o sistema inteiro deixar de funcionar sem apresentar nenhum erro.

O passo 5 é tolerante a falha: se a geração falhar, o alerta é aberto mesmo assim, com a conta. Perder a redação é degradação; perder o alerta seria falha.

## Fluxo de leitura — o painel



O navegador **nunca** fala com o Notion nem com o Gemini. Um front-end que chamasse a API do Notion diretamente precisaria embarcar o token no bundle, e bundle é público por definição — a chave estaria a um "ver código-fonte" de qualquer visitante, com acesso a tudo que a integração enxerga.

## Agendamento

A varredura roda por linha de comando ou pelo disparo manual do workflow em GitHub Actions. **O agendamento por cron está desligado**, e a ausência é uma decisão de segurança, não um item pendente.

Ligá-lo exigiria guardar o token do Notion nos segredos de um repositório público. Sendo um Personal Access Token, ele carrega o acesso do emissor ao workspace inteiro — a limitação descrita na seção 5. Replicar uma credencial de alcance largo em mais um sistema, para ganhar execução automática num trabalho acadêmico, é uma troca que não se paga.

O gatilho fica versionado e comentado no arquivo do workflow, com a condição para religá-lo escrita ao lado: trocar o PAT por uma integração interna escopada à página-raiz. Registrar a condição junto do código é o que faz a decisão sobreviver a quem vier depois; um `schedule` apagado sem explicação seria religado no primeiro impulso.

## 5. Estratégia de autenticação e segurança

### Três credenciais, três escopos

| Credencial      | Onde vive                             | Alcance                            |
|-----------------|---------------------------------------|------------------------------------|
| Token do Notion | segredo da Edge Function e do Actions | tudo que o usuário emissor enxerga |
| Chave do Gemini | segredo da Edge Function e do Actions | cota do projeto                    |
| JWT do usuário  | navegador, emitido pelo Supabase      | a própria sessão                   |

As duas primeiras nunca chegam ao navegador. A terceira nunca chega ao Notion.

### O escopo do token do Notion é maior do que este documento afirmava

Uma versão anterior descrevia o alcance da primeira linha como "só as páginas compartilhadas". **Estava errado**, e a correção importa mais que o erro.

O que revelou foi uma chamada trivial:

```
GET /v1/users -> 403
{"code":"restricted_resource",
"message":"Personal access tokens cannot list users."}
```

O token não é o de uma integração interna: é um **Personal Access Token**. Um PAT age *como o usuário que o emitiu*, com as permissões dele sobre o workspace inteiro. Não há escopo por compartilhamento de página — o escopo é a conta.

Três consequências, em ordem de gravidade:

**Alcance.** O token guardado como segredo desta aplicação acadêmica alcança, em princípio, tudo que a conta emissora enxerga no workspace corporativo, inclusive páginas que nada têm a ver com o projeto. A "fronteira de dados" declarada na página-raiz é uma convenção do código — a varredura e o painel só tocam as três bases — e não uma restrição imposta pela credencial. Convenção de código protege contra engano, não contra abuso.

**Atribuição.** Toda ação da aplicação aparece no Notion como ação do usuário, não de um bot. O histórico da página não distingue o que a automação escreveu do que a pessoa escreveu, o que enfraquece a auditoria do lado do Notion.

**Notificação.** Foi a consequência que apareceu primeiro, e por um caminho enviesado: nenhum dos sete alertas notificou ninguém. A hipótese inicial — "o Notion não notifica ações de integração" — estava errada. A causa real são dois fatos somados, ambos verificados:

1. O PAT age como o próprio usuário, e o destinatário configurado era esse mesmo usuário. **Ninguém é notificado da própria ação.**
2. A página-raiz foi criada sem página-mãe, o que a torna uma página privada de workspace. Uma menção a outra pessoa também não notifica, porque essa pessoa não tem acesso à página.

O diagnóstico só fechou ao inspecionar `created_by` de um comentário criado via API e ver que era o usuário, não o bot. Vale registrar o método: a hipótese plausível e errada sobreviveu a dois testes; o que a derubou foi olhar o dado de atribuição em vez de observar o efeito.

**Correção para produção**, é pré-requisito, não melhoria: criar uma integração interna dedicada, compartilhar apenas a página-raiz com ela, e mover a raiz para um teamspace com os destinatários dos alertas. As três consequências acima caem juntas — o escopo passa a ser real, a atribuição passa a ser do bot, e a notificação passa a ter ator e destinatário distintos.

Notificação, registre-se, **não é requisito do enunciado**, que pede automação. A automação existe e funciona. O que este documento corrige é uma afirmação sobre uma capacidade acessória, e uma afirmação sobre escopo de credencial — a segunda sendo a que realmente importa.

## Autenticação separada de autorização

Existir em `auth.users` prova identidade, não permissão. Um convite aceito por engano cria usuário autenticado. A liberação passa pela tabela `perfis`, com padrão `false`, escrita apenas pelo service role — não

há política de RLS de `insert` ou `update`, porque liberar o próprio acesso não pode ser operação do cliente.

Usuário autenticado e não liberado recebe **403, não 401**. A distinção é funcional: 401 mandaria o cliente para a tela de login num laço que nunca resolve, já que a credencial está correta.

## Superfície reduzida por desenho

Uma única Edge Function, um único método (POST), e um despacho por campo `acao` contra um mapa fechado de cinco rotas. Ação desconhecida devolve 400 antes de qualquer acesso a dado.

Erros retornam mensagem genérica ao cliente. O corpo de erro do Notion nomeia propriedades e identificadores internos, e isso não é informação que o navegador precisa.

## Verificação, não afirmação

As garantias acima foram exercitadas contra o ambiente real, e não só projetadas. Com um JWT de usuário legítimo, tentando contornar a Edge Function e falar direto com o PostgREST:

| Alvo                                  | Resultado               | Leitura                                        |
|---------------------------------------|-------------------------|------------------------------------------------|
| <code>acessos</code>                  | 200 []                  | RLS sem política: zero linhas visíveis         |
| <code>perfis</code>                   | 200 com a própria linha | a política faz o que promete                   |
| <code>alunos</code> (tabela do Lumen) | 403                     | sem grant para o papel autenticado             |
| <code>rpc/criar_perfil</code>         | 404                     | a revogação de EXECUTE removeu a função da API |

O caso de `acessos` merece precisão: a resposta é 200 com lista vazia, não 403. Não é erro — é como RLS funciona. A tabela existe, a requisição é válida, e não há nenhuma linha que o usuário tenha direito de ver. O efeito prático é o desejado, e descrevê-lo como "acesso negado" seria impreciso.

A sequência de autorização também foi verificada na ordem: usuário recém-criado recebeu 403 com a mensagem própria; depois do `update` de liberação, 200. A mesma credencial, resultados diferentes — que é a demonstração de que autenticação e autorização são camadas separadas de fato, e não só no texto.

## O que a aplicação não faz

Não envia e-mail, não expõe endpoint de escrita ao cliente, não aceita identificador de aluno arbitrário sem autenticação, e não tem cadastro aberto. Cada uma dessas ausências é uma classe inteira de ataque que não precisa ser mitigada porque a capacidade não existe.

## 6. Armazenamento e manipulação dos dados

### Onde cada coisa mora

| Dado                | Onde                             | Por quê                             |
|---------------------|----------------------------------|-------------------------------------|
| Cadastro do aluno   | Notion · Alunos                  | é o banco no-code; fonte única      |
| Regras              | Notion · Playbook                | editável por quem é dono da regra   |
| Alertas             | Notion · Alertas                 | a fila onde o trabalho é feito      |
| Contas de acesso    | Supabase <code>auth.users</code> | gerido pelo provedor                |
| Trilha de auditoria | Supabase <code>acessos</code>    | precisa ser inacessível ao auditado |

Nenhum dado de aluno é replicado no Postgres. O único cache é o do Playbook, em memória, com TTL de 60 segundos — curto de propósito, porque a promessa do projeto é que editar a regra mude o comportamento sem deploy, e um cache longo transformaria "sem deploy" em "sem deploy, mas espere uma hora".

### Manipulação

**Conversão de tipos.** O Notion representa cada tipo de propriedade com uma forma diferente de JSON. Toda conversão passa por um único módulo ( `_shared/notion.mjs` ), compartilhado entre Node e Deno. O cliente foi escrito à mão em vez de usar a biblioteca oficial exatamente por isso: a mesma conversão precisa valer nos dois runtimes, e uma dependência a menos é um caminho a menos para eles divergirem.

**Paginação obrigatória.** O Notion devolve no máximo 100 itens por requisição. A leitura percorre todas as páginas. Uma varredura que lesse só a primeira deixaria de gerar alerta para quem estivesse na segunda — silenciosamente.

**Nulo não é zero.** Todo contador pode vir nulo, e nulo significa "não informado", nunca zero. Um aluno sem turma não reprovou no piso de presença: ele não tem piso a cumprir ainda. Tratar nulo como zero produziria uma negativa falsa. A avaliação devolve `elegivel: null` e marca o caso como indeterminado, e a interface é instruída a desenhar barra vazia com o rótulo "não informado" em vez de barra em 0%.

**Limites do formato.** Rich text no Notion tem teto de 2000 caracteres por bloco, e o texto que estoura é sempre o gerado pelo modelo — ou seja, falha em produção e nunca em teste. Texto gerado é recortado; regra do Playbook é **partida em vários blocos**, nunca truncada, porque truncar apagaria em silêncio o final da regra, que costuma ser justamente a exceção.

**Truncamento não é sucesso.** A primeira execução real da varredura gravou sete alertas cujas leituras terminavam no meio da frase — "A coordenação não deve conceder a extensão gratuita de 30 dias de". A causa não era o limite do Notion. O `maxOutputTokens` do Gemini é **compartilhado com o raciocínio interno** do modelo, que neste caso consumiu 383 dos 400 tokens do orçamento e deixou 13 para a resposta:



```
finishReason: MAX_TOKENS
thoughtsTokenCount: 383    candidatesTokenCount: 13
```

Duas correções, e a segunda importa mais que a primeira. A primeira foi aumentar o orçamento — desligar o raciocínio via `thinkingConfig.thinkingBudget: 0` foi testado e devolve resposta vazia neste modelo, então não é alternativa.

A segunda foi passar a **tratar finishReason diferente de STOP como falha**. O código original lia o texto e, encontrando string não vazia, considerava a geração bem-sucedida. Meia frase é pior que nenhuma frase num campo que a coordenação lê como orientação, porque parece completa — e o alerta ia para o Notion sem nenhum sinal de que faltava conteúdo. A degradação declarada — *"(leitura não gerada — consulte a conta que sustenta o alerta)"* — é honesta; a truncada não é.

O episódio reforça a decisão de arquitetura da seção 2: a conta determinística estava correta nos sete alertas, porque não depende do modelo. Só a redação falhou.

### Dados sintéticos verificáveis

Gerados por script versionado, com PRNG de semente fixa. O script é público justamente como prova de que a base é **gerada**, não extraída.

O determinismo é parcial e a exceção é deliberada: as **datas** são ancoradas no dia da execução, não numa data fixa. Custou a reprodutibilidade byte a byte, e a razão é que a automação varre "acesso vencendo em 30 dias" — com datas congeladas, os casos viram "vencido há três meses" em algumas semanas, e a demonstração deixa de exercitar a regra que o projeto existe para demonstrar.

Seis perfis são obrigatórios e cada um exercita uma regra: elegível, negado por piso isolado, muito atrasado, sem turma, turma futura, e certificação travada. Se um deles sair da base, a regra correspondente deixa de ser demonstrável.

## 7. LGPD, ética e governança

---

### Base legal e escopo

Na fase acadêmica **não há tratamento de dado pessoal**: os titulares não existem. Isso não é isenção — é o que torna a fase acadêmica o momento certo de decidir a arquitetura de proteção, antes que dado real esteja em jogo.

O projeto tem como destino declarado tornar-se uma ferramenta operacional real. A promoção exige, no mínimo: base legal definida para o tratamento, revisão do escopo do token, e retirada do rótulo sintético da página-raiz. Essas condições estão escritas na própria página-raiz no Notion, e não só neste documento — um requisito que só existe no documento da faculdade não vai ser lido por quem fizer a promoção.

## Minimização por desenho

O esquema carrega apenas campos que respondem alguma pergunta do escopo. Documento de identificação, telefone e endereço ficaram de fora — **nem sintéticos**, porque nenhuma resposta depende deles e um campo que existe sintético é um campo que alguém preenche com dado real na promoção.

Classificação, considerando a fase de produção:

| Categoria         | Exemplos                 | Natureza     |
|-------------------|--------------------------|--------------|
| Identificação     | nome, e-mail             | dado pessoal |
| Vínculo acadêmico | turma, formação, jornada | dado pessoal |
| Progresso         | presença, módulos, prazo | dado pessoal |

Nenhum campo é dado pessoal sensível na acepção do art. 5º, II. Progresso acadêmico é dado pessoal comum: revela desempenho, não origem racial, convicção religiosa, opinião política, filiação sindical, dado de saúde, vida sexual, genético ou biométrico.

## A fronteira de integração, e por que ela é frágil

As três bases vivem sob uma única página-raiz, e a integração é compartilhada só com ela.

Essa fronteira precisa ser declarada como frágil, porque é: **o escopo de integração do Notion falha aberto**. Basta alguém compartilhar outra página com a mesma integração para ampliar o alcance do token, sem aviso, sem log visível para quem opera a aplicação e sem que nada no código mude.

O contraste com a decisão equivalente do projeto anterior é instrutivo. Lá, a base de conhecimento inteira estava no `.gitignore`, porque ignorar um diretório inteiro **falha fechado** — a exceção precisa ser escrita para o dado vazar. Aqui não existe equivalente que falhe fechado: o Notion não oferece "integração restrita a esta subárvore, permanentemente". A mitigação disponível é documental, e por isso o aviso está na página-raiz, onde quem for compartilhar uma página nova tem chance de vê-lo.

Registrar essa limitação vale mais do que uma afirmação de isolamento que a ferramenta não sustenta.

## Auditoria

Toda consulta grava quem consultou, qual ação e qual identificador. **Não grava o conteúdo devolvido** — duplicaria o dado no log e alargaria exatamente a superfície que a minimização tenta reduzir.

A tabela não tem política de leitura pelo cliente, em nenhuma hipótese. Quem é auditado não lê nem edita a própria trilha. O registro acontece **depois** do sucesso da operação: registrar antes produziria log de consulta que falhou, e "quem viu o quê" deixaria de ser verdade.

**Ruído automático degrada a trilha, e a trilha mostrou isso sobre si mesma.** Durante uma sessão de edição da interface, a tabela registrou uma rajada de 30 a 36 chamadas por minuto, sustentada por cinco minutos — contra as poucas unidades por minuto que uma pessoa usando o painel produz. Em pouco mais de dez minutos a tabela passou de dezenas para 205 registros.

A primeira leitura foi de que a interface tinha um laço de atualização automática. **Estava errada, e o modo como caiu vale mais que o achado.** Duas evidências a derrubaram: a rajada *parou sozinha* — um temporizador não para enquanto a aba está aberta — e o código-fonte da interface não tem `refetchInterval`, `setInterval`, nem chave de consulta instável. A causa provável é o ambiente de edição recarregando a prévia a cada reconstrução.

O que fica estabelecido, e é suficiente para a decisão de projeto: **volume automático de qualquer origem degrada a trilha de auditoria.** Ela existe para responder *quem viu o quê*; afogada em centenas de chamadas que ninguém pediu, o registro de uma consulta deliberada fica indistinguível do ruído em volta.

A resposta certa não é filtrar na gravação — isso esconderia o volume em vez de explicá-lo, e daria à trilha uma seleção que ninguém auditou. É manter a atualização explícita no cliente e **medir**: com o painel aberto e parado, a contagem tem que ficar em zero. A verificação está no `docs/PROMPT_LOVABLE.md`.

Registro do erro: a hipótese do laço foi publicada antes de eu ler o código-fonte da interface, que só entrou no repositório depois. Era uma inferência a partir do efeito, apresentada como diagnóstico.

## Ética da automação

**Alerta não é decisão.** Cada alerta carrega a conta determinística, a leitura gerada, a regra que o originou, e um aviso de que a leitura é sugestão de modelo de linguagem e que a conta prevalece em caso de divergência. Um humano com nome e responsabilidade decide.

**Confiança da fonte é derivada, não declarada.** Regras têm status de revisão, e a confiança é função dele: `Publicado` gera `Alta`, `Para revisar` gera `Média`. Os dois campos não são independentes de propósito — se fossem, uma regra pendente poderia carregar o rótulo "Alta", que é exatamente a mentira que o rótulo existe para impedir.

Uma das cinco regras está deliberadamente `Para revisar`. Alertas dela saem com severidade baixa e com a ressalva no corpo do alerta e no prompt de redação. O propósito não é decorativo: um sistema que trata toda regra com a mesma autoridade ensina o operador a não perguntar de onde a regra veio.

**Ruído é uma falha ética, não só de usabilidade.** Jornadas encerradas — Churn, Abandonou, Reembolsado — são puladas. Cobrar plano de acompanhamento de quem pediu reembolso é o tipo de ruído que faz um painel inteiro ser ignorado, e um painel ignorado deixa de proteger justamente quem ele deveria alcançar.

**Recusa explícita.** O sistema não estima preço, prazo de reembolso ou condição contratual. Esses valores não estão documentados em nenhuma fonte que ele consulta, e estimar seria inventar — sobre dinheiro, para quem tem contrato.

## Anexo — bateria de testes do motor de regras

---

`npm run testar`, sem rede e sem credencial. Dezesseis casos, dos quais o determinante é o seguinte:

```
nega por piso isolado, mesmo com a soma das fases acima de 60%  
CARPA 4 de 5, Pilares 2 de 7 -> elegivel: false  
reprovados: ["Mentorias de Pilares"]
```

Esse caso existe porque a implementação errada — somar as duas fases e comparar com 60% — passa em todos os outros e falha só nele.

Última execução: 21 de 21.